# AWS IAM User + Bedrock Titan Embeddings – Student Notes

Nov 6, 2026 · @VPro Skills

## 1. The big picture (what we built)

We did two things in AWS:

1. **Created an IAM user** – a separate login identity (not the root account) that our code or CLI uses.
2. **Gave that user permission to use Amazon Bedrock** – specifically the **Titan Text Embeddings** model, so our program can turn text into numbers (embeddings).

**Simple analogy:** AWS is a big office building. The **root account** is the building owner's master key – too powerful to carry around. **IAM** is the security desk. An **IAM user** is an employee ID card. A **policy** is the list of rooms that card can open. **Bedrock** is one room (the AI room), and **Titan Embeddings** is one machine inside it.

Flow in one line: **Root account → IAM → Create user → Attach Bedrock policy → Create access keys → Configure CLI/Python → Call Titan Embeddings.**

## 2. Key terms (learn these first)

| Term | Meaning in simple words |
|---|---|
| AWS | Amazon's cloud – rent computers, storage, AI etc. over the internet |
| Root user | The email you signed up with. Full power. Use only for billing/emergencies |
| IAM | Identity and Access Management – controls *who* can do *what* |
| IAM User | A named identity (e.g. `bedrock-dev`) with its own password and/or access keys |
| IAM Group | A collection of users that share the same permissions |
| Policy | A JSON document listing allowed/denied actions |
| Access Key ID + Secret Access Key | Username + password for code/CLI (not for the website) |

| Term | Meaning in simple words |
|---|---|
| Amazon Bedrock | AWS service that gives ready-made AI models through an API |
| Foundation model (FM) | A large pre-trained AI model (Titan, Claude, Llama…) |
| Titan Text Embeddings | Amazon's model that converts text → a list of numbers (vector) |
| Embedding / Vector | Numbers that capture the *meaning* of text. Similar meaning → similar numbers |
| Region | Physical AWS location (e.g. `us-east-1` N. Virginia, `ap-south-1` Mumbai). Models are available per region |
| Model ID | Exact name used in code, e.g. `amazon.titan-embed-text-v2:0` |

## 3. Why embeddings matter

Computers don't understand words – they understand numbers. An embedding model reads a sentence and outputs, for example, 1024 numbers.

- "I love dogs" and "I adore puppies" → vectors that are **close** to each other.
- "I love dogs" and "Stock market fell" → vectors that are **far apart**.

**Used for:** semantic search, chatbots over your own documents (RAG), recommendations, finding duplicates, clustering.

Titan models you may see:

- `amazon.titan-embed-text-v2:0` – newer, default 1024 dimensions (can choose 256 / 512 / 1024), cheaper. **Recommended.**
- `amazon.titan-embed-text-v1` – older, 1536 dimensions.

## 4. Part A – Create the IAM user (console steps)

1. Sign in to the AWS Console (as root or an admin).
2. Search **IAM** in the top search bar → open it.
3. Left menu → **Users** → **Create user**.
4. **User name:** e.g. `bedrock-user`.

5. (Optional) Tick **Provide user access to the AWS Management Console** if this person also needs the website. For code-only use, leave it unticked.
6. Click **Next** → you reach **Set permissions** (Part B).
7. Review → **Create user**.

**Tip:** Best practice is to create a **group** (e.g. `BedrockDevelopers`), attach the policy to the group, then add users to the group. Easier to manage when there are many users.

## 5. Part B – Give Bedrock permissions

On the **Set permissions** screen choose **Attach policies directly** (or add to a group). Two options:

### Option 1 – Easy (for learning)

Search and tick **`AmazonBedrockFullAccess`** (AWS managed policy). This allows everything in Bedrock. Fine for practice, too wide for real projects.

### Option 2 – Least privilege (for real projects)

Create your own policy that allows *only* calling Titan Embeddings:

IAM → **Policies** → **Create policy** → **JSON** tab → paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeTitanEmbeddings",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel"
      ],
      "Resource": "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-
embed-text-v2:0"
    },
    {
      "Sid": "ListModels",
      "Effect": "Allow",
      "Action": [
        "bedrock:ListFoundationModels",
        "bedrock:GetFoundationModel"
      ],
```

```
      "Resource": "*"
    }
  ]
}
```

Name it e.g. `TitanEmbeddingsInvokeOnly` → **Create policy** → attach it to the user.

### How to read a policy (exam favourite)

- **Version** – always `"2012-10-17"` (policy language version, not today's date).
- **Effect** – `Allow` or `Deny`.
- **Action** – what can be done, format `service:Action` (e.g. `bedrock:InvokeModel`).
- **Resource** – on what. The ARN `arn:aws:bedrock:<region>::foundation-model/<model-id>` points to one model. Note the empty account field (`::`) – AWS-owned models have no account number.
- **Rule:** everything is denied by default; an explicit **Deny** always beats an Allow.

**Change the region** in the ARN if you use another one (e.g. `ap-south-1`).

## 6. Part C – Create access keys and configure the CLI

Access keys let your laptop/code act as this user.

1. IAM → **Users** → click `bedrock-user` → **Security credentials** tab.
2. **Access keys** → **Create access key**.
3. Use case: **Command Line Interface (CLI)** (or Local code) → tick the confirmation → **Next** → **Create access key**.
4. **Copy or download the .csv now** – the Secret Access Key is shown only **once**.

Install AWS CLI v2, then run:

```
aws configure
# AWS Access Key ID:     AKIA....
# AWS Secret Access Key: ********
# Default region name:   us-east-1
# Default output format: json
```

This saves keys in `~/.aws/credentials` and region in `~/.aws/config`.

Check who you are logged in as:

```
aws sts get-caller-identity
```

It should show `...:user/bedrock-user`. If it shows root or another user, your keys are wrong.

## 7. Part D – Model access in Bedrock

- **Today:** AWS now enables serverless foundation models (including Amazon Titan) automatically in supported regions. Access is controlled by **IAM permissions** – which is exactly why Part B matters.
- **Older tutorials** show a **Bedrock → Model access → Manage model access** page where you tick Titan Embeddings and click Save. If your console still shows that page, enable the model there.
- Some third-party models (e.g. Anthropic Claude) may ask for a one-time use-case form.
- Check the model exists in your region:

```
aws bedrock list-foundation-models --by-provider amazon \
  --query "modelSummaries[?contains(modelId,'embed')].modelId"
```

## 8. Part E – Test it

### Test with AWS CLI

```
aws bedrock-runtime invoke-model \
  --model-id amazon.titan-embed-text-v2:0 \
  --body '{"inputText":"Hello from AWS Bedrock"}' \
  --cli-binary-format raw-in-base64-out \
  --region us-east-1 \
  output.json
```

Open `output.json` – you'll see `"embedding": [0.012, -0.034, ...]` and `inputTextTokenCount`.

**Note:** the service for *calling* models is **`bedrock-runtime`**; the service for *listing/managing* models is **`bedrock`**. Students mix these up often.

### Test with Python (boto3)

```
pip install boto3
```

```
import boto3, json

client = boto3.client("bedrock-runtime", region_name="us-east-1")

body = {
    "inputText": "I love learning AWS",
    "dimensions": 1024,      # 256, 512 or 1024 (v2 only)
    "normalize": True
}

response = client.invoke_model(
    modelId="amazon.titan-embed-text-v2:0",
    body=json.dumps(body),
    contentType="application/json",
    accept="application/json"
)

result = json.loads(response["body"].read())
vector = result["embedding"]
print("Length:", len(vector))   # 1024
print("First 5:", vector[:5])
```

### Bonus – compare two sentences

```
def embed(text):
    r = client.invoke_model(modelId="amazon.titan-embed-text-v2:0",
                            body=json.dumps({"inputText": text, "normalize":
True}))
    return json.loads(r["body"].read())["embedding"]

a, b = embed("I love dogs"), embed("I adore puppies")
similarity = sum(x * y for x, y in zip(a, b))   # cosine similarity (vectors
are normalized)
print(round(similarity, 3))   # close to 1 = similar meaning
```

## 9. Common errors and fixes

| Error | Why it happens | Fix |
|---|---|---|
| `AccessDeniedException ... not authorized to perform: bedrock:InvokeModel` | User has no Bedrock policy, or the ARN region/model in the policy doesn't match | Attach the policy; make sure region and model ID in the ARN match your call |

| Error | Why it happens | Fix |
|---|---|---|
| `AccessDeniedException: You don't have access to the model` | Model not enabled in this region (older console) | Bedrock → Model access → enable Titan Embeddings |
| `ValidationException: The provided model identifier is invalid` | Typo in model ID or model not in that region | Use exact `amazon.titan-embed-text-v2:0`; check region with `list-foundation-models` |
| `Unable to locate credentials` | `aws configure` not done | Run `aws configure` or set `AWS_PROFILE` |
| `InvalidClientTokenId` / `SignatureDoesNotMatch` | Wrong or deleted access key | Create a new key, re-run `aws configure` |
| `Invalid base64` (CLI) | CLI v2 expects base64 body | Add `--cli-binary-format raw-in-base64-out` |
| `Could not connect to the endpoint URL` | Wrong region name or service (`bedrock` vs `bedrock-runtime`) | Use `bedrock-runtime` for invoke, correct region |

## 10. Security best practices

- Never use the **root** account for daily work; turn on **MFA** for root and admin users.
- Give **least privilege** – only the actions and models needed.
- **Never** put access keys in code or push them to GitHub. Use `aws configure`, environment variables, or (on AWS servers) **IAM Roles** instead of keys.
- Rotate (replace) access keys regularly; delete unused keys.
- Set a **billing alarm / budget** – Bedrock charges per token.

## 11. Quick revision

- IAM user = identity; policy = permissions; access keys = credentials for code.
- `AmazonBedrockFullAccess` = easy; custom policy with `bedrock:InvokeModel` on one model ARN = secure.
- Titan Embeddings v2 model ID: `amazon.titan-embed-text-v2:0`, output 1024 numbers by default.
- Call models with **`bedrock-runtime`**, manage/list with **`bedrock`**.

- Region must match everywhere: CLI config, code, and policy ARN.

## Practice questions

1. Why shouldn't you use the root user for API calls?
2. What is the difference between an IAM user and an IAM role?
3. Write the IAM action needed to call a Bedrock model.
4. What does an embedding represent, and give two use cases.
5. You get `AccessDeniedException` even though the policy is attached. Name two things to check.

**Answers:** (1) Root has unlimited power – a leaked key means full account loss. (2) A user is a permanent identity with long-term keys; a role is assumed temporarily by services/people and gives short-term credentials. (3) `bedrock:InvokeModel`. (4) The meaning of text as numbers – semantic search, RAG chatbots. (5) Region in the ARN vs the call region, and the exact model ID / model access enabled.